

## Data Structures & Algorithms Revision Sheet

This note is designed for rapid recall of sorting boundaries, algorithm selection trees, and shortest-path graph characteristics.

### [OS] 1. Sorting Algorithms Boundary Analysis

Different sorting algorithms exhibit distinct behavior based on input characteristics.

#### 1. Counting Sort: Non-Comparison Linear Bounds

Counting Sort is an integer-sorting algorithm that counts distinct key occurrences and computes prefix sums to place elements.

Standard Complexities:

Time Complexity:  $\Theta(n + k)$

Space Complexity:  $O(n + k)$  (requires  $O(k)$  count array and  $O(n)$  output array)

Where  $n$  is the number of input elements and  $k$  is the range of keys ( $\text{max} - \text{min} + 1$ ).

The Unusual Case (Quadratic Degradation):

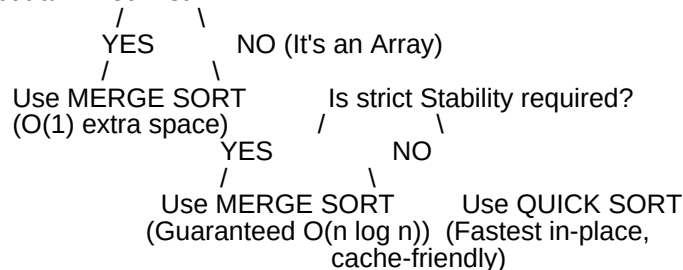
If the key range  $k$  is exponentially larger than  $n$  (e.g.,  $k = O(n^2)$ ), the time complexity degrades to  $O(n^2)$  and the space requirements become massive.

Example: Sorting an array of size  $N = 100$  where elements range from  $0$  to  $10,000$  requires a count array of size  $10,000$ . If elements range up to  $1,000,000$ , it becomes highly inefficient compared to a standard  $O(n \log n)$  comparison sort!

#### 2. Merge Sort vs. Quick Sort Decision Guide

Choosing between these two divide-and-conquer giants depends on your data structures and system constraints:

Is the input a Linked List?



Property

Merge Sort

Quick Sort

Worst-Case Time

$O(n \log n)$  (Guaranteed)

$O(n^2)$  (Unbalanced pivots)

Average-Case Time

$O(n \log n)$

$O(n \log n)$  (Usually faster in practice)

Stability

Stable (Preserves duplicate order)

Unstable

Auxiliary Space

$O(n)$  extra memory (Not in-place)

$O(\log n)$  recursive stack (In-place)

Hardware Pref

Sequential access (Great for Disk/Tape)

Random access (Great for RAM / Cache)

Best Choice For

Linked Lists (no extra array copy needed)  
Arrays (highly cache-friendly)

### 3. Graph Shortest-Path & MST Matrix

A summary of graph algorithm parameters for quick decision-making:

Algorithm  
Type  
Complexity  
Negative Edges?  
Negative Cycles?  
Core Description

Dijkstra  
Greedy  
 $O(E \log V)$  (with Min-Heap)  
No (Fails)  
No  
Single-Source Shortest Path. Works on both directed and undirected graphs.

Bellman-Ford  
DP  
 $O(V \cdot E)$   
Yes (Handles)  
Detects  
Single-Source Shortest Path. Detects negative cycles if relaxation succeeds in the  $V$ -th pass.

Floyd-Warshall  
DP  
 $O(V^3)$   
Yes (Handles)  
No (Fails)  
All-Pairs Shortest Path. Uses three nested loops:  $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ .

Kruskal's  
Greedy  
 $O(E \log E)$   
Yes  
Yes  
Minimum Spanning Tree (MST). Sorts all edges, adds the smallest edge if it doesn't form a cycle. Uses Disjoint Set Union (DSU).

Prim's  
Greedy  
 $O(E \log V)$   
Yes  
Yes  
Minimum Spanning Tree (MST). Grows the tree node-by-node from a starting vertex.

### DSA Common Traps

Dijkstra on Directed/Undirected: Remember that Dijkstra works perfectly on both directed and undirected graphs. Its only limitation is negative edge weights.

Counting Sort Stability: Counting sort is stable only if the final placement loop scans the input array from right to left (or backward) to preserve the relative order of identical keys.

Floyd-Warshall Diagonals: If Floyd-Warshall runs and any diagonal element  $d[i][i]$  becomes negative, it proves the graph contains a negative-weight cycle accessible from node  $i$ .